



E-Commerce Customer Analysis



Project Overview

This markdown cell introduces the project's focus on e-commerce data analysis

The analysis will examine three key areas:

1. Sales trends - to understand revenue patterns over time
2. Customer behavior - to identify purchasing patterns and preferences
3. Profitability insights - to determine which products/customers drive profit

The ultimate purpose is to provide actionable insights for business decisions



Business Problem

This section defines the core business challenge being addressed

E-commerce companies collect extensive transaction data

The challenge is extracting meaningful insights from this raw data

Without proper analysis, businesses cannot identify:

- Profitable customer segments
- High-performing products
- Emerging market trends

E-commerce businesses generate large volumes of transactional data, but without analysis it is difficult to identify profitable segments, products, and trends.

```
In [18]: import pandas as pd
import plotly.express as px # Data visualization library
import plotly.graph_objects as go # Graphical visuals
import plotly.io as pio # Graph template
import plotly.colors as colors
pio.templates.default="plotly_white"
```

```
data=pd.read_csv('superstore.csv',encoding= 'latin- 1') data.head()
```

Dataset Overview

The dataset contains order-level e-commerce data including sales, profit, category, sub-category, customer segment, and order date.

```
In [5]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10800 entries, 0 to 10799
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Row ID                10800 non-null  object
1   Order ID              10800 non-null  object
2   Order Date            9994 non-null   object
3   Ship Date             9994 non-null   object
4   Ship Mode              9994 non-null   object
5   Customer ID           9994 non-null   object
6   Customer Name         9994 non-null   object
7   Segment                9994 non-null   object
8   Country                9994 non-null   object
9   City                  9994 non-null   object
10  State                  9994 non-null   object
11  Postal Code            9983 non-null   float64
12  Region                 9994 non-null   object
13  Product ID             9994 non-null   object
14  Category                9994 non-null   object
15  Sub-Category           9994 non-null   object
16  Product Name           9994 non-null   object
17  Sales                   9994 non-null   float64
18  Quantity                9994 non-null   float64
19  Discount                9994 non-null   float64
20  Profit                  9994 non-null   float64
dtypes: float64(5), object(16)
memory usage: 1.7+ MB
```

```
In [7]: data['Order Date']=pd.to_datetime(data['Order Date'])
        data['Ship Date']=pd.to_datetime(data['Ship Date'])
```

```
In [8]: data.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10800 entries, 0 to 10799
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Row ID                 10800 non-null  object
1   Order ID               10800 non-null  object
2   Order Date             9994 non-null   datetime64[ns]
3   Ship Date              9994 non-null   datetime64[ns]
4   Ship Mode              9994 non-null   object
5   Customer ID            9994 non-null   object
6   Customer Name          9994 non-null   object
7   Segment                9994 non-null   object
8   Country                9994 non-null   object
9   City                   9994 non-null   object
10  State                  9994 non-null   object
11  Postal Code            9983 non-null   float64
12  Region                 9994 non-null   object
13  Product ID             9994 non-null   object
14  Category               9994 non-null   object
15  Sub-Category           9994 non-null   object
16  Product Name           9994 non-null   object
17  Sales                  9994 non-null   float64
18  Quantity               9994 non-null   float64
19  Discount               9994 non-null   float64
20  Profit                 9994 non-null   float64
dtypes: datetime64[ns](2), float64(5), object(14)
memory usage: 1.7+ MB

```

```

In [10]: data['Order Month']=data['Order Date'].dt.month
         data['Order Year']= data['Order Date'].dt.year
         data['Order by Week']= data['Order Date'].dt.dayofweek

```

```

In [11]: data.head()

```

Out[11]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	
0	1	CA-2017-152156	2017-11-08	2017-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Hei
1	2	CA-2017-152156	2017-11-08	2017-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Hei
2	3	CA-2017-138688	2017-06-12	2017-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	.
3	4	US-2016-108966	2016-10-11	2016-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Lau
4	5	US-2016-108966	2016-10-11	2016-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Lau

5 rows × 24 columns



```
In [13]: data['Order Month'] = data['Order Month'].astype('Int64')
data['Order Year'] = data['Order Year'].astype('Int64')
data['Order by Week'] = data['Order by Week'].astype('Int64')
```

```
In [14]: data.head()
```

Out[14]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	
0	1	CA-2017-152156	2017-11-08	2017-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Hei
1	2	CA-2017-152156	2017-11-08	2017-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Hei
2	3	CA-2017-138688	2017-06-12	2017-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	.
3	4	US-2016-108966	2016-10-11	2016-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Lau
4	5	US-2016-108966	2016-10-11	2016-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Lau

5 rows × 24 columns



In [15]: data.info()

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10800 entries, 0 to 10799
Data columns (total 24 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   Row ID                10800 non-null object  
 1   Order ID              10800 non-null object  
 2   Order Date            9994 non-null  datetime64[ns]
 3   Ship Date             9994 non-null  datetime64[ns]
 4   Ship Mode             9994 non-null  object  
 5   Customer ID           9994 non-null  object  
 6   Customer Name         9994 non-null  object  
 7   Segment               9994 non-null  object  
 8   Country               9994 non-null  object  
 9   City                  9994 non-null  object  
10   State                 9994 non-null  object  
11   Postal Code           9983 non-null  float64  
12   Region                9994 non-null  object  
13   Product ID            9994 non-null  object  
14   Category              9994 non-null  object  
15   Sub-Category          9994 non-null  object  
16   Product Name          9994 non-null  object  
17   Sales                  9994 non-null  float64  
18   Quantity              9994 non-null  float64  
19   Discount              9994 non-null  float64  
20   Profit                 9994 non-null  float64  
21   Order Month            9994 non-null  Int64  
22   Order Year             9994 non-null  Int64  
23   Order by Week          9994 non-null  Int64  
dtypes: Int64(3), datetime64[ns](2), float64(5), object(14)
memory usage: 2.0+ MB

```

Exploratory Data Analysis (EDA)

Exploratory analysis is performed to identify patterns and trends in sales, profit, and customer behavior.

```

In [16]: sales_by_month = data.groupby('Order Month')['Sales'].sum().reset_index()
         sales_by_month

```

Out[16]:

	Order Month	Sales
0	1	94924.8356
1	2	59751.2514
2	3	205005.4888
3	4	137762.1286
4	5	155028.8117
5	6	152718.6793
6	7	147238.0970
7	8	159044.0630
8	9	307649.9457
9	10	200322.9847
10	11	352461.0710
11	12	325293.5035

```
In [20]: fig=px.line(sales_by_month,  
                    x='Order Month',  
                    y='Sales',  
                    title='Monthly Sales Analysis')  
fig.show()
```



```
In [21]: data.head()
```


Out[21]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	
0	1	CA-2017-152156	2017-11-08	2017-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Hel
1	2	CA-2017-152156	2017-11-08	2017-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Hel
2	3	CA-2017-138688	2017-06-12	2017-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	
3	4	US-2016-108966	2016-10-11	2016-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Lau
4	5	US-2016-108966	2016-10-11	2016-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Lau

5 rows × 24 columns



 Sales by Category

This analysis compares total sales across different product categories.

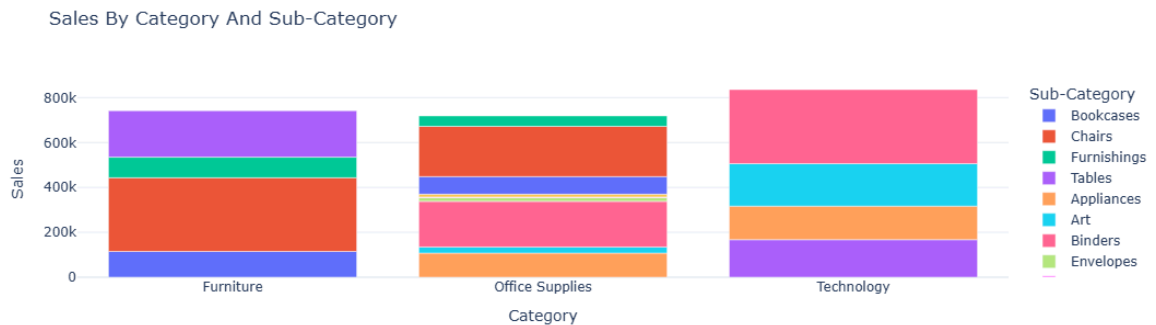
```
In [22]: sales_by_category= data.groupby('Category')['Sales'].sum().reset_index()
fig=px.line(sales_by_category,
            x= 'Category',
            y= 'Sales',
            title= 'Category Analysis')
fig.show()
```



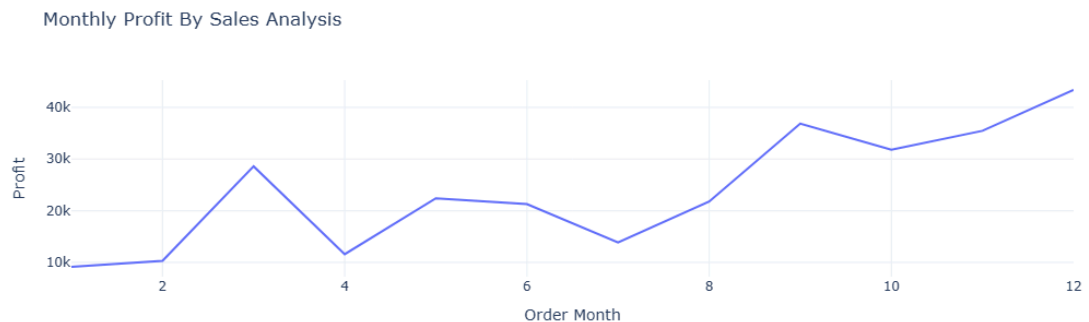
Insight:

Technology and Office Supplies categories contribute the highest sales, indicating strong demand in these segments.

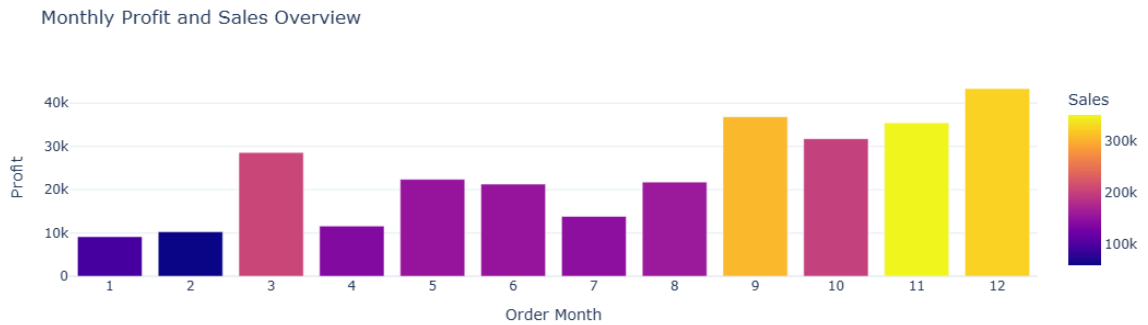
```
In [29]: sales_by_sub_category= (data.groupby(['Category', 'Sub-Category'])['Sales'].sum())
fig=px.bar(sales_by_sub_category,
           x= 'Category',
           y= 'Sales',
           color= 'Sub-Category',
           title= 'Sales By Category And Sub-Category')
fig.show()
```



```
In [31]: monthly_profit_of_sales= data.groupby('Order Month')['Profit'].sum().reset_index
fig=px.line(monthly_profit_of_sales,
            x= 'Order Month',
            y= 'Profit',
            title= 'Monthly Profit By Sales Analysis')
fig.show()
```



```
In [38]: monthly_profit_sales= data.groupby('Order Month')[['Sales', 'Profit']].sum().reset_index
fig=px.bar(monthly_profit_sales,
           x= 'Order Month',
           y= 'Profit',
           color= 'Sales',
           title= 'Monthly Profit and Sales Overview')
fig.update_xaxes(
    tickmode='linear',
    tick0= 1,
    dtick= 1
)
fig.show()
```



```
In [49]: sales_profit_seg = data.groupby('Segment')[['Sales', 'Profit']].sum().reset_index()
fig = px.bar(sales_profit_seg,
             x='Segment',
             y='Profit',
             color='Sales',
             title='Profit Analysis by Customer Segment (Sales Intensity)')
fig.show()
```



```
In [52]: sales_profit_segment = data.groupby('Segment').agg({'Sales': 'sum', 'Profit': 'sum'})
sales_profit_segment['Sales_to_profit_Ratio'] = sales_profit_segment['Sales'] / sales_profit_segment['Profit']
sales_profit_segment[['Segment', 'Sales_to_profit_Ratio']]
```

```
Out[52]:
```

	Segment	Sales_to_profit_Ratio
0	Consumer	8.659471
1	Corporate	7.677245
2	Home Office	7.125416



Key Insights

- Consumer segment generates the highest profit
- Some sub-categories show negative profit despite high sales
- Profitability varies significantly across months



Business Recommendations

- Focus marketing on high-profit customer segments
- Optimize discounts for loss-making products

- Improve inventory planning based on monthly trends

Conclusion

This analysis provides actionable insights into customer behavior, product performance, and profitability, enabling data-driven decision-making for e-commerce businesses.